

# **Professional RAG PDF Assistant**

## **Objective**

This project implements a Retrieval-Augmented Generation (RAG) chatbot that allows a user to upload a PDF, ask natural-language questions about its contents, and receive answers that are strictly grounded in that document. The system cites the source pages behind every answer and returns an explicit fallback message when a question cannot be answered from the uploaded document, so it never fabricates information not present in the source.

## **System Architecture**

The pipeline follows a standard RAG flow, implemented end to end in both a Gradio application (app.py) and a companion Jupyter notebook used for step-by-step development and inspection:

### PDF Upload

1. Text Extraction (pypdf)
2. Text Cleaning (whitespace/null-byte normalization)
3. Text Chunking (page-aware, overlapping windows)
4. Embeddings (SentenceTransformer: all-MiniLM-L6-v2)
5. Vector Storage (ChromaDB, cosine similarity space)
6. Query -> Semantic Retrieval (top-k relevant chunks)
7. Context Assembly (retrieved chunks + document overview)
8. LLM Answer Generation
9. Answer

## **Key Design Decisions**

- **Page-wise chunking:** I split the PDF page by page so that the chatbot can give the correct page number when showing sources.

- **Strict answers from the PDF:** The chatbot is instructed to only use information from the uploaded document and not make up answers. If the information isn't there, it gives a clear fallback message.
- **Distance threshold:** I added a relevance check so that weak or unrelated search results are filtered out instead of being sent to the AI.
- **Document overview:** I included the document's introductory information to help the chatbot answer general questions like “*What is this document about?*” more accurately.
- **Retry for temporary errors:** If Gemini temporarily has a server problem, the system retries the request instead of immediately showing an error.

## Tech Stack

PDF parsing (pypdf)

Embedding model (all-MiniLM-L6-v2, SentenceTransformers)

Vector database. (ChromaDB)

LLM Google Gemini (gemini-3.6-flash)

UI (Gradio)

## Limitations

- **Very general questions:** For broad questions like “*What is this document about?*”, the chatbot mainly uses the introduction of the document. If the introduction does not clearly explain the purpose, the chatbot may say “**I could not find this information in the uploaded document.**”
- **Different TOP\_K values:** The notebook uses **TOP\_K = 8**, while the app uses **TOP\_K = 4**. The app uses fewer results to make responses faster and reduce API usage, while the notebook retrieves more results for testing.
- **Gemini free-tier limit:** Gemini 3.6 Flash has a limited free-tier quota. During development or repeated testing, you may use up the daily limit. If that happens, the app can show a **429**

**RESOURCE\_EXHAUSTED** error until the quota resets. This is a Google API limitation, not a problem with the application.

## Conclusion

The **Professional RAG PDF Assistant** meets all the main requirements of the assignment, including PDF upload, text extraction, chunking, vector storage, searching relevant information, generating answers from the document, showing sources, and providing a fallback response when information is not found. The main risk is **not the project design**, but the Gemini API. During development, the available models and free-tier limits changed.